SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	U.Kavya	**Reg. No.:**	192472114
Date:	22-10-2026	**Duration:**	60 minutes

Code of Conduct

*I ____**U.Kavya (192472114)**_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____U.Kavya___

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.

4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description
   - Scope of the System
   - Functional Requirements
   - Non-Functional Requirements
   - Use Case Descriptions
   - Data Requirements
   - External Interface Requirements
   - System Features
5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement	SRS is well-structured, complete, professionally	SRS is organized with minor missing	SRS includes only essential	SRS is incomplete,

Specification (SRS) Documentation (15%)	organized, and includes all required sections.	sections or formatting issues.	sections with limited detail.	poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional and non-functional requirements for a Container Security and Runtime Behaviour Monitoring Platform. It describes how the platform will assess container images before deployment, validate configurations, monitor running workloads, detect suspicious activity, integrate security checks into CI/CD pipelines, and support incident response and audit.

1.2 Intended Audience

The document is intended for the project team, software developers, DevOps engineers, security analysts, platform administrators, testers, and academic evaluators.

1.3 Definitions and Acronyms

Term	Meaning
Container image	Packaged filesystem and metadata used to create a container.
CVE	Common Vulnerabilities and Exposures identifier for a publicly known vulnerability.
CI/CD	Continuous Integration / Continuous Delivery or Deployment.
Runtime behaviour	Actions performed by a container while it is running.
Orchestrator	Platform that schedules and manages containers, such as Kubernetes.
Security gate	Automated rule that allows, warns about, or blocks a pipeline/deployment.
RBAC	Role-Based Access Control.
SRS	Software Requirements Specification.

2. Scenario Study and Problem Description

2.1 Business Domain

The system belongs to cloud-native application security, DevSecOps, container security, and workload monitoring. It applies security controls across the container lifecycle, from image creation and release through production execution.

2.2 Current Challenges

- Container images may contain outdated packages, vulnerable dependencies, malware, or unnecessary components.
- Misconfigured containers may run as root, use excessive privileges, expose sensitive ports, or mount sensitive host paths.
- A secure image can still be misused after deployment; unexpected processes, network connections, file changes, or privilege changes may indicate compromise.
- Security checks may be separated from CI/CD, allowing risky images to reach production.
- Logs and alerts can be distributed across registries, pipelines, nodes, and orchestration systems, making investigation difficult.
- Teams need traceable evidence of scans, policy decisions, alerts, and response actions for audit.

2.3 Proposed Solution

The proposed platform combines image vulnerability scanning, configuration policy validation, runtime event collection, rule-based and anomaly-based detection, CI/CD security gates, orchestration integration, centralized logging, dashboards, and incident-management workflows.

2.4 Objectives

- Identify known vulnerabilities in container images and their dependencies before release.
- Detects risky container and orchestration configurations before deployment.
- Monitor runtime events and flag suspicious or policy-violating behaviour.

- Prevent images that fail configured security policies from progressing through CI/CD.
- Provide a centralized view of workloads, scan results, alerts, incidents, and audit events.
- Support scalable deployment and repeatable security enforcement.

3. Scope of the System

3.1 In Scope

- Register or connect container image registries and CI/CD pipelines.
- Scan images and dependencies and record vulnerability severity and scan time.
- Validate image metadata and deployment configuration against configurable policies.
- Collect runtime events from supported container hosts or orchestration environments.
- Detect suspicious behaviour using configurable rules and optional baseline/anomaly logic.
- Generate alerts, group related events into incidents, and record investigation and response actions.
- Apply configurable CI/CD gate outcomes: pass, warn, or block.
- Integrate with container orchestration and centralized log storage.
- Provide role-based dashboards, reports, and audit history.

3.2 Out of Scope for Initial Version

- Replacing a full enterprise SIEM, endpoint detection platform, or vulnerability-management program.
- Guaranteeing detection of every zero-day vulnerability or unknown attack.
- Automatically deleting or isolating production workloads without an explicitly configured and authorized response policy.
- Providing a new container runtime or orchestrator.

3.3 System Boundary

The platform consumes image metadata/layers, configuration files, pipeline events, orchestration metadata, and runtime telemetry. It produces scan reports, policy decisions, alerts, incidents, dashboards, audit records, and optional approved response requests.

4. Stakeholders and Actors

Stakeholder / Actor	Type	Interest / Responsibilities
Developer	Primary	Builds images, reviews findings, fixes vulnerable dependencies and configuration issues.
DevOps / CI-CD Engineer	Primary	Configures pipeline integration and security gates; manages deployment workflows.
Security Analyst	Primary	Investigates alerts, prioritizes risks, manages incidents, and reviews audit evidence.
Platform Administrator	Primary	Manages users, roles, policies, integrations, retention, and system health.
Container Orchestrator	Secondary / external system	Provides workload inventory, deployment metadata, and supported runtime events.
Image Registry	Secondary / external system	Stores container images and provides image references or scan targets.
CI/CD Platform	Secondary / external system	Triggers scans, receives gate results, and records pipeline status.
Central Logging / Notification Service	Secondary / external system	Receives logs or forwards alerts through configured interfaces.
Auditor / Compliance Reviewer	Supporting user	Reviews historical scans, policy changes, incidents, and response evidence.

5. Functional Requirements

The following requirements use unique identifiers to support testing and traceability. “Shall” indicates a mandatory requirement for the proposed system.

ID	Requirement area	Requirement statement
FR-01	Image registration	The system shall allow authorized users to register a registry, image repository, tag, or immutable image digest for assessment.
FR-02	Image vulnerability scanning	The system shall scan supported container images for known operating-system and application dependency vulnerabilities.
FR-03	Finding details	For each vulnerability finding, the system shall store the affected package, installed version, vulnerability identifier when available, severity, evidence, and remediation guidance when available.
FR-04	Scan history	The system shall retain timestamped scan results and associate them with an image digest and scan-tool/database version where available.
FR-05	Configuration validation	The system shall evaluate container and orchestration configuration against configurable security policies.
FR-06	Policy checks	The system shall support checks such as privileged mode, root execution, host namespace use, risky capabilities, sensitive host mounts, exposed ports, and missing resource limits where applicable.
FR-07	Runtime telemetry	The system shall collect supported runtime events, including process activity, file activity, network activity, and security-relevant container metadata.
FR-08	Suspicious activity detection	The system shall evaluate runtime events using configurable rules and, where enabled, behaviour baselines or anomaly scoring.
FR-09	Alert generation	The system shall create alerts containing time, workload/container identity, severity, detection reason, and relevant event evidence.

FR-10	Alert management	Authorized analysts shall be able to acknowledge, assign, comment on, change status, and close alerts or incidents.
FR-11	CI/CD integration	The system shall expose a supported integration method, such as CLI, API, or pipeline plugin, to trigger scans and retrieve results.
FR-12	Security gate	The system shall return, pass, warn, or block decisions according to configurable thresholds and policy rules.
FR-13	Orchestration integration	The system shall integrate with at least one supported container orchestrator to discover workloads and associate events with namespaces, pods, services, or equivalent resources.
FR-14	Centralized logging	The system shall forward or store security events, scan results, policy decisions, and incident actions in a searchable central log store.
FR-15	Dashboard	The system shall display image risk, policy violations, running workload inventory, alerts, incidents, and trend summaries.
FR-16	Reporting	The system shall generate exportable reports for scans, vulnerability status, policy compliance, incidents, and audit activity.
FR-17	Incident response	The system shall document response steps and, where integrations permit, initiate authorized actions such as requesting workload isolation or stopping a deployment.
FR-18	Audit trail	The system shall record security-relevant actions, including login events, policy changes, scan requests, gate decisions, incident updates, and administrative changes.
FR-19	Identity and access	The system shall authenticate users and enforce role-based permissions for operational and administrative functions.
FR-20	Notifications	The system shall support configurable notifications for high-priority findings and incidents through at least one configured notification channel.

FR-21	Policy management	Authorized administrators shall be able to create, update, enable, disable, and version security policies.
FR-22	Retention and search	The system shall support searching and filtering findings and events by time range, severity, image, workload, environment, and status, subject to permissions.
FR-23	Health monitoring	The system shall expose health/status information for scanners, runtime collectors, integrations, and background jobs.
FR-24	Failure handling	The system shall record failed scans or unavailable telemetry sources and shall not silently treat missing results as a successful security check.

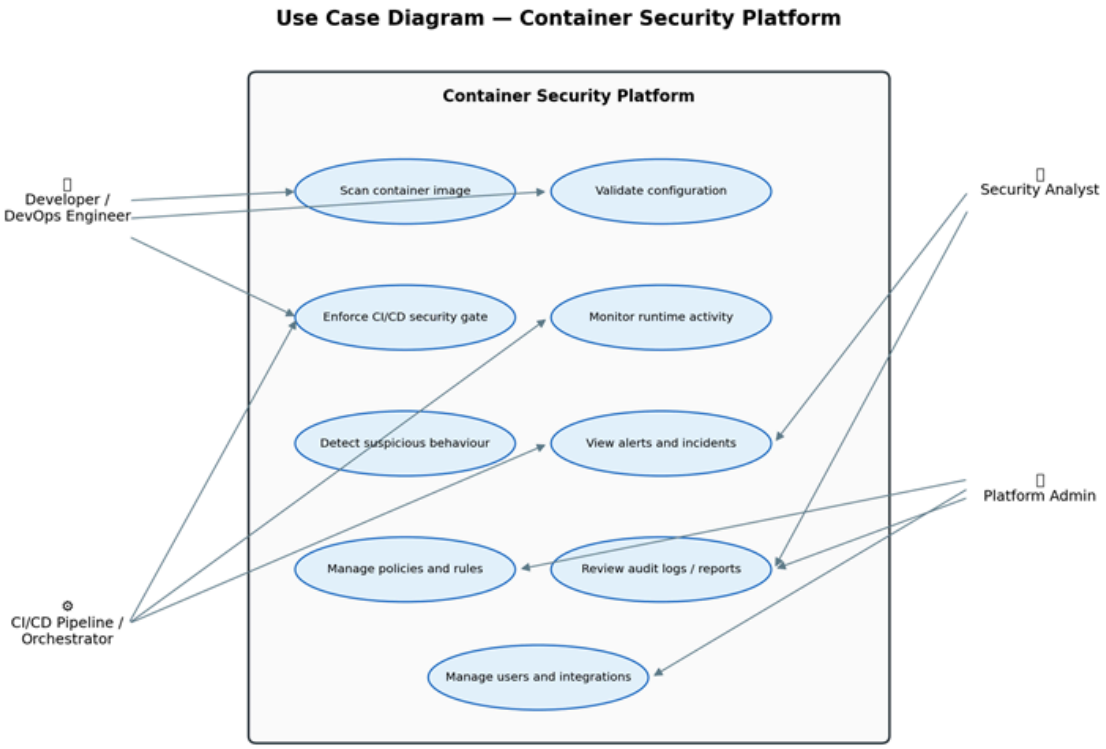
6. Non-Functional Requirements

ID	Quality attribute	Requirement
NFR-01	Performance	For the agreed reference environment, dashboard queries and common API requests should complete within 3 seconds at the 95th percentile, excluding external scanner execution time.
NFR-02	Scan latency	The system shall show scan status and final results; target scan duration shall be benchmarked against representative image sizes and documented in the deployment plan.
NFR-03	Security	All user and service access shall be authenticated; sensitive communications shall use TLS; secrets shall be stored using an approved secret-management mechanism.
NFR-04	Authorization	The system shall enforce least privilege and role-based access control for dashboards, policies, incident actions, and administrative settings.
NFR-05	Reliability	Transient failures in integrations or background jobs shall be logged and retried according to configurable retry policies without duplicating incident records unnecessarily.

NFR-06	Availability	The target service availability shall be 99.5% per month for the production deployment, excluding planned maintenance, subject to infrastructure support.
NFR-07	Scalability	The architecture shall support adding scanner workers and runtime collectors as the number of images, clusters, and events increases.
NFR-08	Usability	A security analyst shall be able to locate a high-severity alert and view its supporting evidence from the dashboard without needing direct database access.
NFR-09	Maintainability	Components shall be modular, documented, configurable, and testable; policy rules and integrations should be updateable without redesigning the entire platform.
NFR-10	Interoperability	The platform shall use documented APIs and supported formats/protocols for integration with registries, CI/CD systems, orchestrators, and logging services.
NFR-11	Auditability	Audit records shall include actor/service identity, timestamp, action, target, and outcome where applicable; access to audit records shall be restricted.
NFR-12	Data integrity	Scan results, policy decisions, and incident histories shall be associated with stable identifiers and protected against unauthorized modification.
NFR-13	Privacy	The system shall collect only runtime and identity data needed for security monitoring and shall document data retention and access rules.
NFR-14	Portability	The application components should be deployable in a documented containerized environment; supported operating systems and orchestrators shall be listed.
NFR-15	Recoverability	Backup and restoration procedures shall be documented and tested for configuration, policies, incident data, and audit records.

NFR-16	False-positive handling	Detection rules shall be tunable and alerts shall include enough evidence for analysts to validate or dismiss a detection.

7. Use Case Diagram and Descriptions



7.1 Major Use Cases by Actor

Actor	Major use cases
Developer	Submit image for scan; review findings; update image/configuration; inspect pipeline decision.
DevOps / CI-CD Engineer	Configure pipeline integration; trigger scan; retrieve security result; enforce gate; review failed pipeline.

Security Analyst	View alerts; investigate evidence; assign/acknowledge incident; record response; close incident; export report.
Platform Administrator	Manage users and roles; configure integrations; manage policies; configure retention; monitor component health.
Orchestrator / Registry / CI-CD	Provide image/workload metadata; send events; receive scan/gate results; support approved response actions.
Auditor	Review audit trail, scan history, policy changes, and incident outcomes.

7.2 Use Case UC-01: Scan Container Image

Primary actor: Developer or CI/CD pipeline

Goal: Determine whether an image contains known vulnerabilities.

Preconditions: The image reference is reachable; the requester is authorized; scanner service is available.

Trigger: A user requests a scan or a pipeline submits an image.

Main flow: 1. The actor submits image repository/tag or digest. 2. The platform validates the request. 3. The scanner retrieves image metadata/layers. 4. The scanner compares package inventory against vulnerability data. 5. The platform stores findings and risk summary. 6. The platform displays results and returns a status.

Alternative flows: If the image cannot be fetched, the scan is marked failed. If vulnerability data is unavailable, the result is marked incomplete or stale rather than clean.

Postconditions: A timestamped scan record is available and linked to the image identity.

Acceptance condition: A test image with a known vulnerable package produces a finding when the scanner's vulnerability feed identifies it.

7.3 Use Case UC-02: Enforce CI/CD Security Gate

Primary actor: CI/CD pipeline

Goal: Prevent release of images that violate release policy.

Preconditions: The pipeline integration and gate policy are configured.

Trigger: A build or release stage requests a security decision.

Main flow: 1. Pipeline submits image identity and environment. 2. Platform obtains or runs required scans. 3. Findings are evaluated against severity thresholds and configuration rules. 4. Platform returns pass, warn, or block with reasons. 5. Pipeline records the decision and continues or stops accordingly.

Alternative flows: If scanning fails or results are stale, the configured fail-open/fail-closed policy applies; the default recommended for production is fail-closed for required checks.

Postconditions: The gate decision and evidence are recorded in the audit trail.

Acceptance condition: An image violating a configured blocking rule receives a block decision and the pipeline can stop release.

7.4 Use Case UC-03: Detect and Investigate Runtime Threat

Primary actor: Security analyst; runtime collector is the event source.

Goal: Identify suspicious workload behaviour and support investigation.

Preconditions: Runtime monitoring is enabled and the workload is covered by a supported collector.

Trigger: A runtime event matches a detection rule or exceeds an anomaly threshold.

Main flow: 1. Collector sends a normalized event. 2. The detection engine evaluates it. 3. The platform creates an alert with workload context and evidence. 4. Analyst opens the alert, reviews related events, and assigns an owner. 5. Analyst records investigation and response. 6. The incident is resolved and retained for audit.

Alternative flows: If event ingestion is delayed, the system records collector health and ingestion delay. If an alert is a false positive, the analyst records the disposition and may request a rule adjustment.

Postconditions: Alert status, investigation notes, and any authorized response actions are auditable.

Acceptance condition: A simulated rule-matching event creates an alert with timestamp, workload identity, severity, and detection reason.

7.5 Use Case UC-04: Manage Security Policy

An authorized administrator creates or edits a policy, specifies scope and severity thresholds, tests it against sample data where supported, publishes a version, and reviews the audit record. Unauthorized users must not be able to modify or publish policies.

8. Data Requirements

8.1 Main Data Entities

Entity	Key fields / attributes	Purpose
User	user_id, username, role, status, last_login	Identity, authorization, and accountability.
Image	image_id, registry, repository, tag, digest, created_at	Stable identity for a container image.
ScanRun	scan_id, image_id, start_time, end_time, status, scanner_version	Tracks scan execution and result freshness.
VulnerabilityFinding	finding_id, scan_id, package, version, CVE, severity, fixed_version, evidence	Records image vulnerability details.

ConfigurationFinding	config_finding_id, target_id, rule_id, severity, evidence, status	Records misconfigurations and policy violations.
Workload	workload_id, cluster, namespace, pod/container identity, image_digest, environment	Connects runtime events to deployed workloads.
RuntimeEvent	event_id, timestamp, workload_id, event_type, process/network/file details, source	Stores normalized runtime telemetry.
Alert / Incident	incident_id, alert_id, severity, status, owner, evidence_refs, notes	Supports detection triage and incident handling.
Policy	policy_id, version, scope, rule_set, threshold, enabled, updated_by	Defines security rules and gate thresholds.
GateDecision	decision_id, pipeline_id, image_digest, outcome, reasons, timestamp	Explains pass/warn/block results.
AuditEvent	audit_id, actor, action, target, outcome, timestamp, source_ip/service	Provides traceability for security-relevant actions.

8.2 Data Validation and Retention

- Each scan and gate decision shall have a unique identifier and timestamp.
- Image findings shall be linked to an immutable image digest when available, not only a mutable tag.
- Timestamps shall use a consistent time standard (UTC recommended) and be rendered in the user's selected time zone.
- Required fields shall be validated at ingestion; malformed events shall be rejected or quarantined with an error record.

- Retention periods shall be configurable by data category and organizational policy. The final values must be agreed before deployment.
- Access to logs and runtime evidence shall be restricted by role; secrets and credentials must not be written into logs.

9. External Interface Requirements

Interface	Requirements
User interface	Web dashboard with authentication, navigation, filters, scan details, policy editor, alerts, incidents, reports, and system health.
CI/CD interface	Documented CLI, REST API, or pipeline plugin to request scans and receive pass/warn/block decisions; support non-interactive service authentication.
Container registry	Integration using supported registry APIs and least-privilege credentials; support image references and digests.
Orchestrator	Integration with a supported orchestrator API or event mechanism to discover workloads and associate runtime telemetry with deployment metadata.
Runtime collector	Host/container-level component or supported telemetry source that emits normalized security events with timestamps and workload identity.
Logging interface	Structured event output (for example JSON) and/or integration with a configured centralized logging system.
Notification interface	Configurable integration with email, chat, or incident-management tools; delivery failures shall be logged.

Authentication interface	Local identity provider or organization-approved SSO/OIDC integration where available.

9.1 Communication and Interface Security

- Use encrypted transport for network communication.
- Use separate service identities and least-privilege credentials for each integration.
- Validate API inputs and authenticate service-to-service calls.
- Rotate credentials and avoid storing secrets in source code or plain-text configuration.

10. System Features and Block Diagrams

10.1 High-Level Architecture

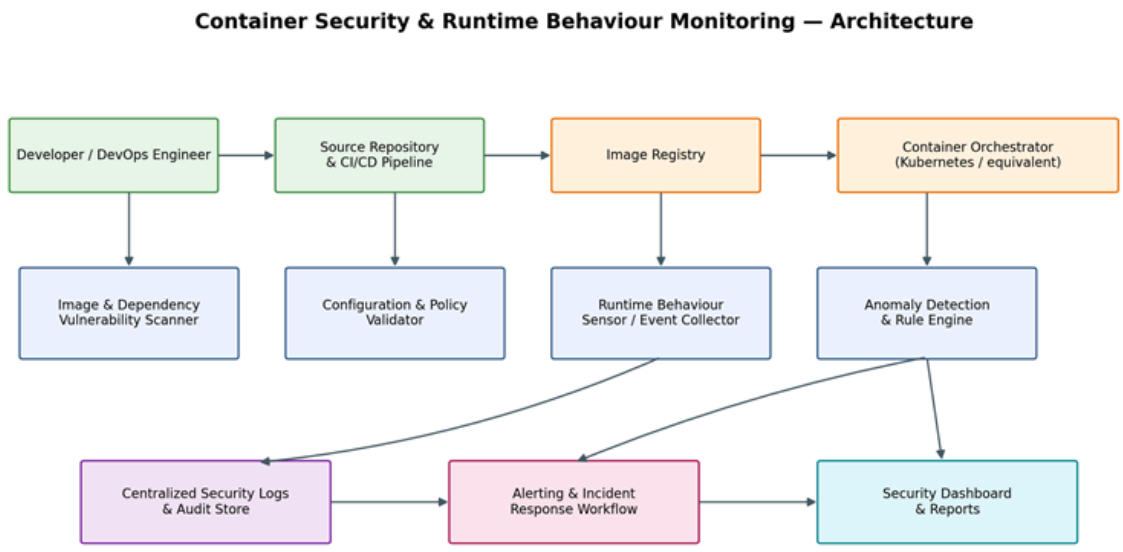


Figure 2.

Architecture explanation: The CI/CD pipeline submits image references for scanning and configuration checks. The policy engine returns a release decision. After deployment, runtime collectors send security events to the detection engine. Findings, alerts, decisions, and audit

events are centralized and presented through dashboards and reports. Analysts can follow approved response workflows.

10.2 Secure Container Lifecycle

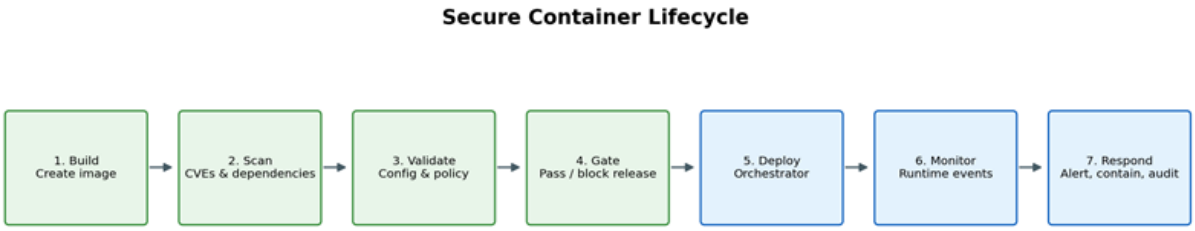


Figure 3.

10.3 Core Feature Summary

Feature	Input	Processing	Output
Image vulnerability scanning	Image reference/digest	Inventory packages and compare with vulnerability data	Scan report, findings, risk summary
Configuration validation	Container/deployment configuration	Evaluate policy rules	Misconfiguration findings and remediation hints
Runtime monitoring	Runtime events and workload metadata	Normalize and evaluate event stream	Security events and workload context
Threat detection	Normalized runtime events	Rules and optional anomaly detection	Prioritized alerts
CI/CD security gate	Scan results and policy	Evaluate thresholds and required checks	Pass / warn / block

Incident response	Alerts, analyst decisions, integrations	Triage, assignment, response tracking	Incident history and audit evidence
Centralized visibility	Scans, events, incidents, audit logs	Index, filter, correlate, summarize	Dashboard, search, and reports

11. Assumptions and Constraints

11.1 Assumptions

- The organization has or can provide access to container registries, a CI/CD platform, and at least one supported orchestration environment.
- The platform can access vulnerability intelligence from a maintained database or scanning engine.
- Runtime telemetry collection is permitted by the organization and supported by the host/container environment.
- Users have assigned roles and follow documented incident-response procedures.
- The organization will define severity thresholds, production gate rules, and data-retention periods.
- The platform is an aid to security operations and does not guarantee that all vulnerabilities or attacks will be detected.

11.2 Constraints

Constraint category	Constraint / impact
Technical	Integration depends on the registry, CI/CD, orchestrator, host OS, runtime, and APIs supported by the chosen tools.
Operational	Runtime monitoring must be tuned to limit performance overhead, alert noise, and disruption to production workloads.

Economic	Scanner licenses, compute/storage, log ingestion, retention, and monitoring infrastructure may affect the project budget.
Legal / compliance	Collection and retention of logs, process details, and network metadata must follow applicable privacy, security, and organizational policies.
Security	Credentials, scanner services, agents, and response actions must themselves be protected from misuse.
Time-related	Implementation scope must prioritize core scanning, configuration checks, CI/CD gating, runtime alerting, and audit logging before optional advanced analytics.
Availability	The platform depends on the availability of registry APIs, vulnerability feeds, CI/CD runners, orchestration APIs, and log services.
Accuracy	Vulnerability feeds can be delayed, and runtime detection can produce false positives or miss unknown behaviours.

12. Requirement Validation and Quality Review

12.1 Completeness Checklist

- Image vulnerability scanning is covered by FR-01 to FR-04.
- Configuration validation is covered by FR-05 and FR-06.
- Runtime monitoring and suspicious activity detection are covered by FR-07 to FR-10.
- CI/CD security gates are covered by FR-11 and FR-12.
- Orchestration, centralized logging, dashboards, and reporting are covered by FR-13 to FR-16.
- Incident response and audit mechanisms are covered by FR-17 and FR-18.

- Identity, policy management, notifications, search, and health checks are covered by FR-19 to FR-24.
- Performance, security, reliability, usability, scalability, maintainability, availability, interoperability, privacy, and recovery are addressed in Section 6.

12.2 Ambiguity, Redundancy, and Consistency Review

Potential issue	Validation result / improvement
"Abnormal behaviour" is broad	Define initial detection rules, severity mapping, supported event types, and measurable alert criteria during design.
"Continuous monitoring" can be interpreted differently	Specify event coverage, collection interval/streaming approach, health reporting, and acceptable ingestion delay for the selected runtime.
"Secure deployment" is not absolute	Use explicit pass/warn/block policies and document accepted residual risk.
Scanner performance varies by image size	Benchmark agreed representative images and document the target environment rather than claiming one universal scan time.
Fail-open vs fail-closed may conflict with availability goals	Define policy per environment; production-required checks should default to fail-closed, with any exception explicitly approved and audited.
Automated response can be risky	Require authorization, safeguards, audit records, and configurable approval for disruptive actions.
Retention requirements may be organization-specific	Set exact retention periods and deletion procedures before implementation.

12.3 Traceability to Problem Statement

Problem statement capability	Requirement coverage
Scan container images for vulnerabilities	FR-01 to FR-04
Validate container configurations	FR-05 to FR-06
Monitor runtime behaviour	FR-07
Detect suspicious or abnormal activities	FR-08 to FR-10
Integrate security gates with CI/CD	FR-11 to FR-12
Support orchestration and logging	FR-13 to FR-14
Incident response and audit mechanisms	FR-17 to FR-18
Improve visibility and scalable deployment	FR-15, FR-16, FR-22, NFR-07, NFR-10

13. Acceptance Criteria

- A supported image can be scanned and the result includes a scan identifier, image digest, status, and findings.
- A known vulnerable test image produces a vulnerability finding when the scanner database contains that vulnerability.
- A deliberately insecure configuration is flagged with a rule identifier and evidence.
- A simulated suspicious runtime event produces an alert with severity, timestamp, workload identity, and detection reason.
- A pipeline receives pass, warn, or block according to configured thresholds; a blocking finding prevents the release stage from proceeding when the gate is enforced.
- An analyst can acknowledge, assign, investigate, and close an incident; all changes appear in the audit trail.
- An administrator can configure a policy and unauthorized users cannot modify it.
- Dashboard filters and report exports return results consistent with the stored records and the user's permissions.
- Integration failures and missing telemetry are visible and are not silently represented as successful security checks.

14. Conclusion

This SRS translates the given scenario into a structured and testable set of requirements for a container security platform. It combines preventive controls—image scanning, configuration validation, and CI/CD gates—with detective and response controls—runtime monitoring, alerting, centralized logging, incident workflows, and audit trails. Before implementation, stakeholders should validate the selected integrations, measurable performance targets, detection rules, response permissions, and retention periods.

Appendix A: Suggested Implementation Components (Illustrative)

Possible implementation choices include a container image scanner, a policy engine, Kubernetes or another supported orchestrator, a runtime event sensor, a message/log pipeline, a relational or document database for findings, and a web dashboard. These are examples only; the SRS does not mandate specific vendors or tools. Final choices should be based on compatibility, licensing, deployment constraints, and project scope.

Appendix B: Suggested Test Data

- A container image containing a package with a known published vulnerability.
- A deployment manifest configured with privileged mode or an unnecessary capability.
- A benign workload that establishes normal runtime behaviour.
- A simulated runtime event that matches a test detection rule.
- A pipeline test in which the image passes, warns, and fails under different policy thresholds.